

Proj4rs - Rust Implementation of PROJ

<https://github.com/3liz/proj4rs>

What is PROJ ?

A library for PROJ is transformation library that transforms geospatial coordinates from one coordinate reference system (CRS) to another. This includes cartographic projections as well as geodetic transformations.

- **Projections**
- **Transformations**
- Implement the ISO 19111:2019 standard
- Definitions (EPSG)

First release in 1970

Maintained by Even Rouault

<https://en.wikipedia.org/wiki/PROJ>

Proj "ecosystem"

Translations to other language

- Javascript: <https://github.com/proj4js>
- Php: <https://github.com/proj4php/proj4php>
- Java: <https://github.com/locationtech/proj4j>
- Go: <https://github.com/go-spatial/proj>

But

- Only the projections/transformations part is concerned so far.
- Transformations using the PROJ v4 model

The 'Why'

The start

- Experiment with WASM
- Actual work on WASM build of the full PROJ library: <https://github.com/OSGeo/PROJ/issues/4327>
- Limit our ambition to our actual needs in Lizmap: a reprojection module to use in OpenLayers as Proj4js replacement.
- Lightweight with the minimal features.

Why Rust ?

- WASM compilation tooling included
- Reusable implementation as binding for C, Python, Php.
- Great Support for JS interoperability (js-sys, web-sys)
- Speed ?

The 'How'

Target WASM from the start

- Minimum dependencies (only 1 required crates)
- Enforce code locality - avoid heap allocation
- Reuse JS context resources

Sync with PROJ implementation

- PROJ is the source of truth
- No a blind transpiling, use idiomatic Rust when rewriting projections.
 - Eventually may lead to some PROJ contribution: <https://github.com/OSGeo/PROJ/issues/4522>
- Require maximum absolute tolerance error - in meters - of 10^{-6}

Features

- Support 20 most common projections and their variants
- Support nvtv2 Nadgrids (TIFF grids planned)
- Light weight WASM (120 Ko of compiled web assembly code)



May be Used as Proj4js replacement in OpenLayers

<https://docs.3liz.org/proj4rs/>

Proj4rs do not:

- Handle authority codes, only projections.
- Handle WKT format or other normalized formats: use companion library <https://github.com/3liz/proj4wkt-rs>

The transformation from EPSG:XXXX to EPSG:YYYY does not work !

i.e : does not return the same result as PROJ

Proj strings

Define an projection and its parameters to (or from) geographic (lat/lon/alt) coordinates.

```
+proj=utm +zone=39 +ellps=GRS80 +towgs84=0,0,0,0,0,0,0 +units=m
```

Projection

Parameters

PROJ 4: Transformations use a pivot WGS84 Datum transform:

Projected -> Geographic -> Cartesian -> Datum transform -> Cartesian -> Geographic -> Projected

In PROJ 4 (and PROJ 5) Datum transforms may no be applied if no Datum is specified for one of the projection definition.

Comparison with PROJ 6+

In PROJ 4 and 5: A transformation is defined by two proj strings.

In PROJ 6: This no more the case, transformations operation use a *transformation pipeline* which generalize the the previous PROJ 4 schema.

From the [PROJ documentation](#):

```
cs2cs has been reworked to use proj_create_crs_to_crs() internally, with late binding capabilities, and thus is no longer constrained to using WGS84 as a pivot (also called as early binding method). When cs2cs of PROJ 6 is used with PROJ.4 expanded strings to describe the CRS, including +towgs84, +nadgrids and +geoidgrids, it will generally give the same results as earlier PROJ versions. When used with AUTHORITY:CODE CRS descriptions, it may return different results.
```

!! Watch out when using proj definitions from [epsg.io](#) with Proj4rs (or Proj4js) !!

What's wrong with EPSG:3857 ?

Proj string definition from PROJ

```
> projinfo epsg:3857 -o PROJ
+proj=merc +a=6378137 +b=6378137 +lat_ts=0 +lon_0=0 +x_0=0 +y_0=0 +k=1 +units=m +nadgrids=@null +wktext +no_defs +type=crs
```

PROJ pipeline from EPSG:31370 to EPSG:3857:

```
+proj=pipeline
+step +inv +proj=lcc +lat_0=90 +lon_0=4.367486666666667 +lat_1=51.16666723333333
+lat_2=49.8333339 +x_0=150000.013 +y_0=5400088.438 +ellps=intl
+step +proj=push +v_3
+step +proj=cart +ellps=intl
+step +proj=helmert +x=-106.8686 +y=52.2978 +z=-103.7239 +rx=-0.3366 +ry=0.457
+rz=-1.8422 +s=-1.2747 +convention=coordinate_frame
+step +inv +proj=cart +ellps=WGS84
+step +proj=pop +v_3
+step +proj=webmerc +lat_0=0 +lon_0=0 +x_0=0 +y_0=0 +ellps=WGS84
```

Is there something wrong ?

What's wrong with EPSG:3857 - Solution

- PROJ use `webmerc` instead of `merc`
- PROJ apply Helmert transform while PROJ 4 schema will not.

Use the following EPSG:3857 definition:

```
+proj=webmerc +ellps=WGS84 +lat_0=0 +lon_0=0 +x_0=0 +y_0=0 +towgs84=0,0,0,0,0,0,0
```

In Proj4rs, defining `+towgs84` will force a Datum transform.

The full story: <https://github.com/3liz/proj4rs/issues/39>

Conclusion

- We have a WASM projection module that works with OL6
- Probably stick to the PROJ 4 transformation pipeline which appears to be **good enough** for a vast majority of use cases.
- Rework some EPSG definitions...
- Tiff gridshift supports

Questions ?

